College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID607001: Introductory Application Development Concepts
Level 6, Credits 15

Project

Assessment Overview

In this **individual** assessment, you will develop a **REST API** using **Express** and **Node.js**, and deploy it as a **web service** on **Render**. You will choose the idea for the **REST API**. It could be related to sport, culture, food or something else you are interested in.

In **development** and **testing**, your data will be stored in a **PostgreSQL** database on **Docker** and in **production**, your data will be stored in a **PostgreSQL** database on **Render**. As you develop your **REST API**, you will need to write **API tests** to verify the functionality of your **REST API** using **Chai** and **Mocha**.

For marking purposes, provide a video demonstrating the functionality of your **REST API** and the **API tests**. Marks will also be given for code quality and best practices, documentation and **Git** usage.

Learning Outcome

At the successful completion of this course, learners will be able to:

1. Design and build secure applications with dynamic database functionality following an appropriate software development methodology.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Practical	20%	Thursday 29th May at 2.59 PM	1
Project	80%	Monday 9th June at 7.59 AM	1

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Monday 9th June** at **7.59 AM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** across all assessments in **ID607001: Introductory Application Development Concepts**.

Submission

You **must** submit all application files via **GitHub Classroom**. Here is the URL to the repository you will use for your submission – https://classroom.github.com/a/EYdgJL1H. If you do not have not one, create a **.gitignore** and add the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore. Create a branch called **project**. The latest application files in the **project** branch will be used to mark against the **Functionality** criterion. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **8.00 AM**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using an **AI generative tool**. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

Learning to use AI tool is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in **ID607001: Introductory Application Development Concepts**.

Instructions

Functionality - Learning Outcome 1 (50%)

- Models (5%)
  - Ten models. Each model must have at least five fields. One model must be the **User** model.
  - A range of data types, i.e., **Int**, **String**, **Boolean**, etc.
  - Five relationships between models, i.e., one-to-one, one-to-many, many-to-many, etc.
  - Three enums. Each enum must have at least three values.
- Authentication and Role-Based Access Control (10%)
  - Three roles, i.e., **SUPER_ADMIN**, **ADMIN** and **NORMAL**.
  - Register a **NORMAL** user, but not a **SUPER_ADMIN** or **ADMIN** user.
  - All users can login.
  - Lock a user for ten minutes after five failed login attempts.
  - All users can logout. When a user logs out, the user's token is blacklisted.
- CRUD Operations (15%)
  - A **repository**, **controller** and **route** file for each **model**. Each **controller** file needs to contain the following CRUD operations: **getAll**, **getById**, **create**, **update** and **delete**.
  - When performing **create** and **update** operations, validate the request body. Each field must have four validation rules.
  - **Filter** and **sort** data using **query parameters**. All **fields** must be filterable and sortable (in ascending and descending order).
  - **Paginate** data using **query parameters**. The default number of data per page is 25.
  - **SUPER_ADMIN** Permissions
    - Can perform the following operations on the **User** model:
      - **getAll** and **getById** operations on all users.
      - Create, update and delete **ADMIN** and **NORMAL** users, but cannot update their **password**.
      - Create a **SUPER_ADMIN** user.
      - Update their data except **role**, but not other **SUPER_ADMIN** users.
      - Cannot delete other **SUPER_ADMIN** users, including themselves.
      - Disable and enable users. When a user is disabled, the user cannot login.
    - Perform CRUD operations on the other models.
  - **ADMIN** Permissions
    - Can perform the following actions on the **User** model:
      - **getAll** and **getById** operations on **ADMIN** and **NORMAL** users, but not **SUPER_ADMIN** users.
      - Create, update and delete **NORMAL** users, but cannot update their **password**.
      - Cannot create, update and delete **SUPER_ADMIN** and **ADMIN** users.
      - Update their data except **role**, but not other **ADMIN** users.
      - Cannot delete other **ADMIN** users, including themselves.
    - Perform CRUD operations on the other models.
  - **NORMAL** Permissions
    - Can perform the following actions on the **User** model:
      - **getAll** and **getById** operations on themselves, but not other users.
      - Update their data except **role**, but not other users.

        - Cannot delete other users, including themselves.
    - Perform **getAll** and **getById** operations on the other models.

- Seeding and API Tests (10%)
  - Seed five **NORMAL** users via a **CSV** file.
  - Seed five **ADMIN** users via a **JSON** file.
  - Seed five **SUPER_ADMIN** users via **GitHub Gist**.
  - Provide the following **API tests**:
    - CRUD operations for each model.
    - Filter and sort data for each model.
    - Pagination for each model.
    - Authentication and role-based access control.
    - Limit the number of requests for CRUD operations.
- Logging and Security (5%)
  - Log all requests and responses in development to a file called **combined.log**.
  - Secure the **X-Powered-By** header.
  - Limit the number of requests for **getAll** and **getById** operations to 20 requests within a 2 minute window. Set the message to **You have exceeded the number of requests: 20. Please try again in 2 minutes.** when the limit is reached.
  - Limit the number of **create**, **update** and **delete** operations to 10 requests within a minute window. Set the message to **You have exceeded the number of requests: 10. Please try again in 1 minute.** when the limit is reached.
- Development, Testing and Production (5%)
  - Environment variable key's are stored in a **.env** file.
  - Use **/api/v1** as the base URL for the **REST API**.
  - Use **Docker** to run the **PostgreSQL** database in development and testing.
  - Use **Render** to run the **PostgreSQL** database in production.
  - Deploy the **REST API** as a **web service** on **Render**.

Code Quality and Best Practices - Learning Outcome 1 (40%)

- Project Structure (10%) - The codebase should be well-organised with a clear and logical structure that separates concerns and promotes reusability and maintainability.
- Naming Conventions (5%) - File, class, function, variable and resource group names should be clear, descriptive and consistent across the codebase.
- Documentation and Comments (5%) - The codebase should be well-documented using the **JSDoc** format with comments that explain complex logic and clarify the purpose of classes, functions or complex sections.
- Code Formatting (5%) - Consistent indentation and spacing should be used across the codebase. It includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections. It should be automated using a tool like **Prettier**.
- Dead Code (5%) - The codebase should not contain unused or redundant files, classes, functions, variables or resource groups. Keeping unnecessary codebase around can lead to confusion, clutter and potential bugs down the line.
- Performance and Scalability (10%) - The codebase should be optimised for performance, using efficient algorithms and data structures to minimise bottlenecks.

Documentation and Git Usage - Learning Outcome 1 (10%)

- **GitHub** issues to help you organise and prioritise your development work. The course lecturer needs to see consistent use of **GitHub** issues for the duration of the assessment.
- Provide the following in your repository **README.md** file:
  - A URL to the **REST API** as a **web service** on **Render**.
  - How to setup the development environment?
  - How to setup the testing environment?
  - How to seed the **PostgreSQL** database?
  - How to run the **API tests**?
  - How do you open **Prisma Studio**?
  - An **ERD** of your **REST API**.
  - A URL to the video demonstrating the functionality of the **REST API** and the **API tests**.
- Use of **Markdown**, i.e., headings, bold text, code blocks, etc.
- Correct spelling and grammar.
- A **.gitignore** file containing the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/Nod
- Your **Git commit messages** should:
  - Reflect the context of each functional requirement change.
  - Be formatted using an appropriate naming convention style.

Additional Information

- Your **REST API** idea must be signed off by the course lecturer before you start your development work.
- If you **do not** provide a video demonstrating the functionality of your **REST API** and the **API tests**, you will receive a mark of **zero** for this assessment.
- **Do not** rewrite your **Git** history. It is important that the course lecturer can see how you worked on your assessment over time.
- You need to show the course lecturer the initial **GitHub** issues before you start your development work. Following this, you need to show the course lecturer your **GitHub** issues at the end of each week.